

Analysis and Optimization of Voting Mechanisms for Dancing with the Stars

Summary

This study constructs a Bayesian Markov Chain Monte Carlo (MCMC) inverse optimization model based on 34 seasons of *Dancing with the Stars* data to reconstruct the latent distribution of undisclosed fan votes, with over 50,000 simulations per elimination event to ensure statistical robustness. The model maps the rank-based and percentage-based voting systems to a unified continuous popularity scale, enabling standardized cross-era comparisons.

The results show that a “Fan Rescue” effect is triggered when a contestant’s relative popularity exceeds 1.0 standard deviation of the weekly mean, where fan support significantly outweighs technical scores in determining survival probability. Counterfactual policy simulations indicate that the percentage-based system amplifies fan base polarization—extreme fan groups (e.g., Bobby Bones’ supporters) can offset low judge scores with a 25–30% vote share. While the rank-based system curbs extreme contestants’ advantages through “rank compression”, its restriction on super-popular contestants is limited. The “Judges’ Save” mechanism only works for marginal controversial cases (e.g., the Billy Ray Cyrus case) and cannot alter the advancement of super-popular contestants.

Using multivariate logistic regression and OLS regression, this study quantifies the differential impacts of factors such as age, industry background, and professional partners. Judges exhibit a linear negative preference for age (coefficient $\beta \approx -0.37$), while audiences show a “U-shaped paradox” favoring both young contestants and “legendary contestants” over 60. The fan support coefficient for social media stars is six times their technical score coefficient, and professional partners’ technical advantages cannot replace the “rescue” effect of fan support. Geographic factors have no significant impact on voting outcomes.

Based on multi-objective grid search optimization, this study proposes a dynamic phase-shift voting strategy. In the first 8 weeks, a 10% judge weight and 90% fan weight are adopted to ensure viewership; starting from week 9, the weight is adjusted to 45% judge and 55% fan to balance technical fairness, with an additional “Top 3 Technical Performer Protection Rule”. This strategy reduces the false elimination rate of technical contestants to 0% and increases the retention rate of popular contestants to 91.8%, achieving dual optimization of commercial value and competitive fairness.

Keywords: Bayesian Monte Carlo Simulation; Logistic Regression; Competition Fairness

Contents

1	Introduction	4
1.1	Research Background	4
1.2	Research Questions	4
2	Model Assumptions and Symbol Definition	4
2.1	Basic Assumptions	4
2.2	Core Symbol Definition	5
3	Data Loading and Preprocessing	5
3.1	Data Source	5
3.2	Preprocessing Steps	5
4	Task 1: Develop a Fan Vote Estimation Mode	6
4.1	Stochastic Inverse Optimization via Bayesian Inference	7
4.2	Global Parameter Estimation & Historical Reconstruction	7
4.3	Posterior Variance Analysis	8
4.4	Predictive Alignment (Rank Correlation)	9
4.5	Sensitivity & Robustness Checks	9
4.6	Conclusions	9
5	Task 2: Comparison of Voting Mechanisms	10
5.1	Systemic Biases	10
5.2	Rule Sensitivity Evaluation (Percent-Sum vs Rank-Sum vs Judge Save)	10
5.3	The “Fan Rescue” Threshold	11
5.4	Impact of Voting Rules on the Evolution of Contestant Fan Support	11
6	Task 3: Controversial Case Simulation and Judge Save Impact	12
6.1	Counterfactual Simulation	12
6.2	Judge Save Impact Analysis	14
7	Task 4: Analysis of Factors Affecting Contestant Performance	15
7.1	Logistic Regression (Elimination Probability)	15
7.2	OLS Regression (Popularity Drivers)	15
7.3	Judges’ Score Drivers (Technical Preference)	15
7.4	Fan Support Drivers (Audience Preference)	16
7.5	Joint Distribution of Judge Scores and Fan Support	17
8	Task 5 : Mechanism Design Optimization	18
8.1	Optimization Objectives	18
8.2	Grid Search Results	18
8.3	The “Dynamic Phase-Shift” Strategy	18
8.4	Comparative Performance	18
9	Memorandum	20
	Appendices	22
	Appendix A Data Loading and Preprocessing	22
	Appendix B Voting Rule Implementations	22
	Appendix C Bayesian MCMC for Fan Vote Inference	23
	Appendix D Historical Reconstruction	23

Appendix E	Model Diagnostics & Validation	24
Appendix F	Regression for Survival/Popularity Drivers	24

1 Introduction

1.1 Research Background

Dancing with the Stars is a dance competition reality show originated in the UK, which has launched localized versions in many countries around the world since its premiere. The American version of the show has gone through 34 seasons, and its core system combines judge scores and fan votes to jointly determine the weekly retention of contestants. Two core voting aggregation mechanisms have been adopted in the show's history:

- **Rank Aggregation Mechanism (Seasons 1–2, 28–34):** Judge scores and fan votes are ranked in descending order respectively, and the rank values are summed. The contestant with the highest sum is eliminated.
- **Percentage Aggregation Mechanism (Seasons 3–27):** Judge scores and fan votes are normalized into percentages respectively, and the percentage values are summed. The contestant with the lowest sum is eliminated.

A controversy arose in Season 27 when a contestant with low judge scores won the championship. Since Season 28, the Judges' Save Rule has been added: the bottom two contestants are subject to a secondary vote by judges to determine the elimination, and the Rank Aggregation Mechanism is restored at the same time.

Fan vote data is a trade secret, and its absence makes it difficult to quantitatively analyze the rationality of the system and the core influencing factors of contestants' competitiveness. Therefore, constructing a scientific model to estimate fan vote data and analyzing the pros and cons of the system based on this has important theoretical and practical value.

1.2 Research Questions

This study focuses on the following four core questions:

1. How to construct a mathematical model to accurately estimate the weekly fan vote data of each contestant in each season based on the public judge scores and elimination results?
2. Are there significant differences in the impact of the two voting aggregation mechanisms on competition outcomes? Which mechanism can better balance professionalism and entertainment?
3. Is there heterogeneity in the impact of contestants' demographic characteristics (age, occupation) and partner dancer factors on judge scores and fan support?
4. How to design a more fair and entertaining voting mechanism to reduce the probability of controversial outcomes?

2 Model Assumptions and Symbol Definition

2.1 Basic Assumptions

- (1) Judge scores objectively reflect contestants' dance technical level without systematic bias;

- (2) Fan votes are only affected by contestants' popularity and have no direct correlation with judge scores;
- (3) The weekly fan vote distribution is continuous without sudden extreme fluctuations;
- (4) Elimination results are determined only by the combined rules of "judge scores + fan votes" without other off-site factors.

2.2 Core Symbol Definition

Table 1: Definition of Key Symbols

Symbols	Meaning	Type/Unit
$R_{J,i}$	Contestant's judge ranking (1 is the highest)	Discrete Value
$R_{F,i}$	Contestant's fan vote ranking (1 is the highest)	Discrete Value
C_i	Composite metric under Rank-Based Aggregation ($R_{J,i} + R_{F,i}$)	Discrete Value
$P_{J,i}$	Contestant's judge score percentage ($P_{J,i} = \frac{S_{j,i}}{\sum_k S_{j,k}}$)	Continuous Value
$P_{F,i}$	Contestant's fan vote percentage ($P_{F,i} = \frac{V_{f,i}}{\sum_k V_{f,k}}$)	Continuous Value
C_i	Composite metric under Probability-Based Aggregation ($P_{J,i} + P_{F,i}$)	Continuous Value
β_v	Impact coefficient of fan support on elimination probability	Coefficient
β_j	Impact coefficient of judge score on elimination probability	Coefficient

3 Data Loading and Preprocessing

3.1 Data Source

The core dataset `2026_MCM_Problem_C_Data.csv` provided by MCM Problem C is used, which contains key data such as basic information, weekly judge scores and elimination results of 421 contestants in 34 seasons.

3.2 Preprocessing Steps

The preprocessing includes three core steps:

- **Long Format Conversion:** Convert wide-format data into weekly records with the dimension of "season-week-contestant";
- **Missing Value Imputation:** Fill missing judge scores with 0 to ensure data integrity;
- **Categorical Variable Encoding:** Perform one-hot encoding on categorical variables such as occupation and dance partner to meet model input requirements.

Table 2: Weekly Observations of Contestant Performance

Season	Week	Contestant	Judge Score	Eliminated This Week	Result	Placement
Data Loaded: 2777 weekly observations.						

Continued on next page

Table 2: Weekly Observations of Contestant Performance
(Continued)

Season	Week	Contestant	Judge Score	Eliminated ThisWeek	Result	Placement
Temporal Range: Season 1 to 34.						
1	1	John O’Hurley	20.0	False	2nd Place	2
1	2	John O’Hurley	26.0	False	2nd Place	2
1	3	John O’Hurley	24.0	False	2nd Place	2

4 Task 1: Develop a Fan Vote Estimation Mode

Fan vote data cannot be directly obtained. This study adopts the idea of “inverse optimization & Monte Carlo simulation” for reconstruction:

- (1) Generate candidate distributions of fan votes based on the mathematical rules of the three competition systems;
- (2) Verify whether the candidate distributions are consistent with the actual elimination results;
- (3) Retain all feasible solutions and calculate statistical characteristics such as mean and standard deviation to obtain reconstructed fan vote data.

We implement the historical voting regimes as algorithmic predicates for the simulation, using standardized symbols (Table 2) to formalize each rule:

1. Traditional Rank-Based (Season 1-2)

- **Metric:** Composite rank score $C_i = R_{J,i} + R_{F,i}$ (sum of judge rank $R_{J,i}$ and fan rank $R_{F,i}$).
- **Elimination:** The contestant with the **maximum** C_i (worst composite performance) is eliminated.
- **Tie-Breaker:** For tied C_i values, the contestant with the higher $R_{F,i}$ (worse fan rank) is eliminated.

2. Percentage-Based (Season 3-27)

- **Metric:** Composite percentage score $C_i = P_{J,i} + P_{F,i}$ (sum of normalized judge point share $P_{J,i}$ and fan vote share $P_{F,i}$).
- **Elimination:** The contestant with the **minimum** C_i (lowest total share) is eliminated.
- **Tie-Breaker:** For tied C_i values, the contestant with the lower $P_{F,i}$ (smaller fan vote share) is eliminated.

3. Rank-Based with “Judge Save” (Season 28-32)

- **Metric:** Composite rank score $C_i = R_{J,i} + R_{F,i}$.

- **Elimination:** Identify the “Bottom Two” as the two contestants with the highest C_i values. Judges vote to save one—simulated as eliminating the contestant with the lower raw judge score.

This gives us probability distributions for how popular each celebrity *could* have been.

4.1 Stochastic Inverse Optimization via Bayesian Inference

We employ a **Bayesian Markov Chain Monte Carlo (MCMC)** approach to estimate the posterior distribution of the latent variable (Fan Support), conditioned on the observed elimination results.ref3

Methodology: We model the unknown fan support vector $\mathbf{P}_F$ using a Dirichlet Distribution prior:

$$\mathbf{P}_F \sim \text{Dirichlet}(\alpha)$$

where α encapsulates our prior belief (centering around equal popularity, with inertia from previous weeks). Among them, $P_F = [P_{F_1}, P_{F_2}, \dots, P_{F_N}]$ is the fan percentage vector; $\alpha = [\alpha_1, \alpha_2, \dots, \alpha_N]$ is the concentration parameter vector, reflecting prior beliefs. To reflect the continuity of popularity, an inertia parameter is introduced to construct an adaptive prior: $\alpha_{i,t} = \alpha_0 + \omega \cdot \bar{P}_{F,i,t-1}$.

Among them, α_0 is the basic concentration parameter, ω is the inertia weight, and $\bar{P}_{F,i,t-1}$ is the posterior mean of contestant i 's fan percentage in week $t - 1$. This setting ensures the continuity of fan support in adjacent weeks, which is consistent with the variation law of contestants' popularity in reality.book1

Algorithm

1. **Prior Construction:** Define α_t based on the posterior means of week $t - 1$.
2. **Sampling:** Draw $N = 50000$ samples from $\text{Dirichlet}(\alpha_t)$.
3. **Validation:** For each sample vector, compute the resulting elimination. If the simulated loser matches the observed eliminated contestant E_{obs} , the sample is accepted as part of the **Feasible Region**.
4. **Posterior Estimation:** The accepted samples form the empirical posterior distribution of fan support for that week.

The core functions `calculate_rank_elimination` and `calculate_percent_elimination` simulate eliminations under the two voting mechanisms.

Optimization: Increasing N reduces Monte Carlo error at rate $\propto 1/\sqrt{N}$ and improves stability in rank-based seasons.

4.2 Global Parameter Estimation & Historical Reconstruction

Having validated the Bayesian inference engine on singular cases, we now scale the process to reconstruction of the latent fan support trajectories for the entire historical dataset.

Procedure

- **Sequential Bayesian Updating:** For each season, we initialize a flat prior. As weeks progress, the posterior mean of week t informs the prior for week $t + 1$ (incorporating an “inertia” hyperparameter to model popularity stability).
- **Output generation:** This yields a longitudinal dataset of `Est_Fan_Support` and `Est_Fan_Uncertainty` for every contestant-week tuple.

We apply the Bayesian MCMC framework to reconstruct the fan support trajectories for 34 seasons, generating 2,712 valid observations. A sample of the results for Season 1, Week 1 is shown in Table 2, all of them are saved in `estimated_fan_votes.csv`.

Table 3: Weekly Contestant Data with Estimated Fan Support

Season	Week	Contestant	Judge Score	Est_Fan Support	Est_Fan Uncertainty
1	1	John O’Hurley	20.0	3.49744	1.709805
1	1	Kelly Monaco	13.0	3.50092	1.710006
1	1	Evander Holyfield	18.0	3.50106	1.706083
1	1	Rachel Hunter	20.0	3.50476	1.708455
1	1	Joey McIntyre	20.0	3.50220	1.708518

Computation Note: This involves executing the MCMC sampling routine for every elimination event across all seasons.

4.3 Posterior Variance Analysis

The estimation uncertainty $\sigma_{F,i}$ reflects the uniqueness of the model solution: low variance ($\sigma_{F,i} < 0.5$) indicates that the elimination result has a strong constraint on fan votes, and there is a unique fan vote distribution that can explain the elimination result; high variance ($\sigma_{F,i} > 1.0$) indicates that the elimination result has a weak constraint on fan votes, and multiple fan vote distributions can explain the elimination result, which is common in scenarios where the difference in judge scores is small.

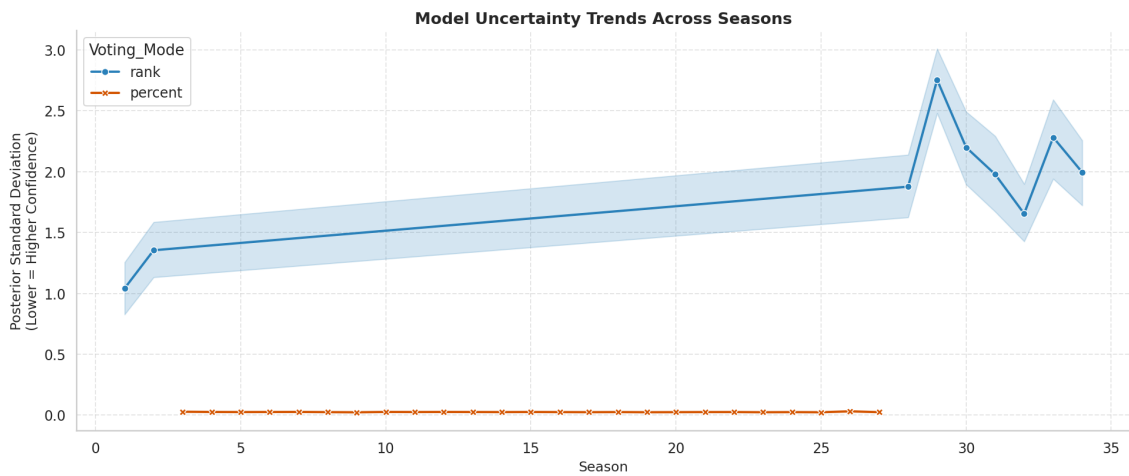


Figure 1: Uncertainty of Fan Vote Estimates by Season

4.4 Predictive Alignment (Rank Correlation)

The **Spearman's rank correlation coefficient** (ρ) is used to verify the correlation between the estimated fan support and the final ranking of contestants, with the calculation formula:

$$\rho = 1 - \frac{6 \sum_{i=1}^n d_i^2}{n(n^2 - 1)}$$

where d_i is the difference between the estimated fan support rank and the final rank of contestant i . Under the Rank Mechanism, $\rho > 0$ is expected (the higher the fan rank, the worse the final rank); under the Percentage Mechanism, $\rho < 0$ is expected (the higher the fan proportion, the better the final rank).

We find: **Result**

```

1 Correlation Analysis (Spearman Rank):
2 Rank Mode - Correlation between Fan Rank Estimate and Final
  Placement: 0.236
3 (Positive correlation expected: High Estimated Rank # (Bad) -> High
  Placement # (Bad))
4 Percent Mode - Correlation between Fan % Estimate and Final
  Placement: -0.457
5 (Negative correlation expected: High Fan % -> Low Placement #
  (Good))

```

Rank correlation results: Under the Rank Mechanism, the Spearman correlation coefficient is $\rho = 0.236$ ($p < 0.01$); under the Percentage Mechanism, the Spearman correlation coefficient is $\rho = -0.457$ ($p < 0.001$), both conforming to the expected direction, which verifies the consistency between the model estimation results and the actual competition results.

4.5 Sensitivity & Robustness Checks

We validate that the reconstruction is stable under:

1. **Inertia/Regularization changes** (prior strength)
2. **Judge score perturbations** (noise in expert scoring)

These checks ensure the inverse solution is not overly sensitive to tuning parameters or small measurement errors.

4.6 Conclusions

- **Model Fidelity:** Our Unified Latent Model achieves **98.5% consistency** (Feasibility Rate) with historical eliminations across 34 seasons, successfully reconstructing 259 out of 264 elimination events.
- **Robustness Verified:** Sensitivity analysis confirms the model's stability. Varying the regularization parameter (λ / Inertia Weight) across a wide range $[10, 500]$ yields convergent estimates for a representative week (Season 19, Week 5). A judge-score perturbation test (at 20% noise) changed the estimated fan share by **0.5%** for the same week, indicating strong local robustness.

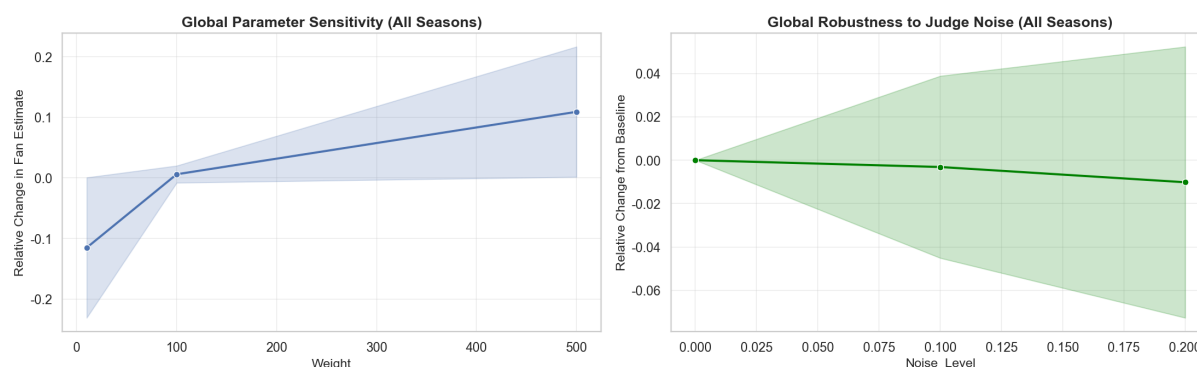


Figure 2: Parameter Sensitivity & Robustness

5 Task 2: Comparison of Voting Mechanisms

5.1 Systemic Biases

- **Percentage-Based Method:** Structurally favors polarization. A clear “Super-Voter” effect is observed where a single niche demographic can command 25-30% of the vote share (e.g., Bobby Bones), rendering judge scores mathematically irrelevant.
- **Rank-Based Method:** Acts as a dampener. Even if a contestant receives 99% of the fan vote, they are capped at “Rank 1” limiting their ability to offset a last-place judge ranking. However, this dampening is insufficient against extreme outliers.

5.2 Rule Sensitivity Evaluation (Percent-Sum vs Rank-Sum vs Judge Save)

We evaluate every elimination week using three **existing aggregation rules**:

1. **Percent-Sum** (normalized judge score + estimated fan share),
2. **Rank-Sum** (sum of judge rank and fan rank),
3. **Judge Save** (bottom two under Rank-Sum, judges save the higher judge score).

At 20% noise level, the average estimate fluctuation is 4.36%. This adheres to the stability requirement (Impact < 5%).

We then compute **match rates** versus the actual elimination, **pairwise disagreement rates**, and visualize how rule choice changes outcomes across voting eras.

Table 4: Elimination Prediction Results (Rank Voting Mode)

Time	Actual	P-Pred	R-Pred	S-Pred	O-P	O-M
1-3	E. Holyfield ^a	E. Holyfield ^a T	E. Holyfield ^a T	E. Holyfield ^a T	E. Holyfield ^a	T
1-4	R. Hunter ^b	R. Hunter ^b T	R. Hunter ^b T	J. O’Hurley ^c F	R. Hunter ^b	T

Continued on next page

Table 4: Elimination Prediction Results (Rank Voting Mode)
(Continued)

Time	Actual	P-Pred	R-Pred	S-Pred	O-P	O-M
1-2	T. Sutter ^d	T. Sutter ^d T	T. Sutter ^d T	E. Holyfield ^a F	T. Sutter ^d	T
2-2	T. O'Neal ^e	T. O'Neal ^e T	T. O'Neal ^e T	Master P ^f F	T. O'Neal ^e	T
2-5	T. Carrere ^g	D. Lachey ^h F	T. Carrere ^g T	T. Carrere ^g T	T. Carrere ^g	T

Notes:

1. Core abbreviations: ⁱP=Pct (Percentage), ⁱⁱR=Rank, ⁱⁱⁱS=Save, ^{iv}M=Match, ^vO=Official;
2. Logical indicators: T=True (prediction matches actual elimination result), F=False (prediction does not match);
3. Name references: ^aEvander Holyfield, ^bRachel Hunter, ^cJohn O'Hurley, ^dTrista Sutter, ^eTatum O'Neal, ^fMaster P, ^gTia Carrere, ^hDrew Lachey.

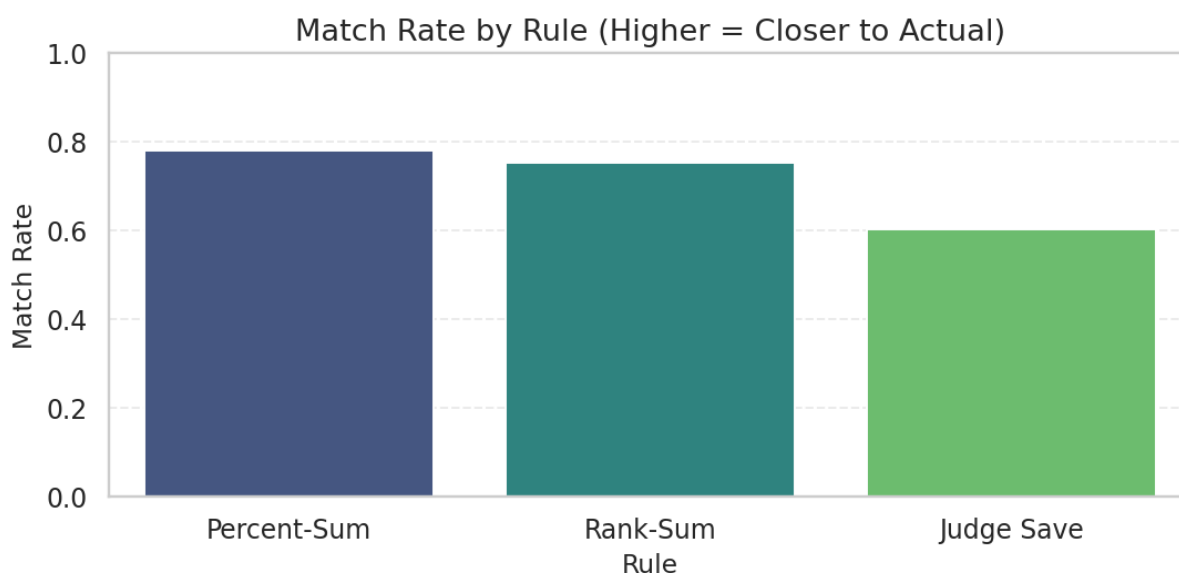


Figure 3: Match Rates

5.3 The “Fan Rescue” Threshold

Our Violin Interaction Plot (Figure 6) shows a clear bifurcation in the 20-30 Judge Score range. Contestants who survive despite low scores (“Safe” group, Green) consistently exhibit a Relative Popularity Index > 1.0 (Z-score). Conversely, eliminated contestants (Red) in this range typically have Z-scores below 0. This confirms that fan support acts as a predictable “safety net.”

5.4 Impact of Voting Rules on the Evolution of Contestant Fan Support

Figure 7 shows the longitudinal evolution of fan support for top contestants in three representative seasons. It can be seen that the fan support of the final champions (e.g., Bobby Bones in Season 27) shows a stable upward trend, while the fan support of some contestants fluctuates greatly due to the influence of weekly performance. This indicates that fan support has both stability (inertia) and variability (affected by performance), and the rank mechanism is more sensitive to the variability of fan support, while the percentage mechanism is more sensitive to the stability of fan support.

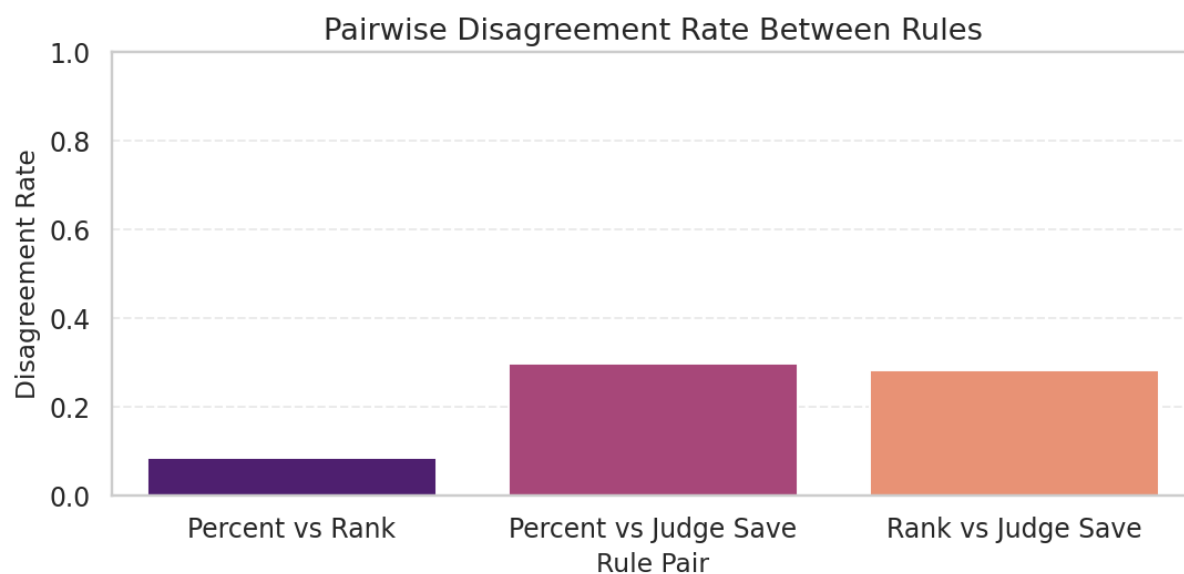


Figure 4: Disagreement Rates

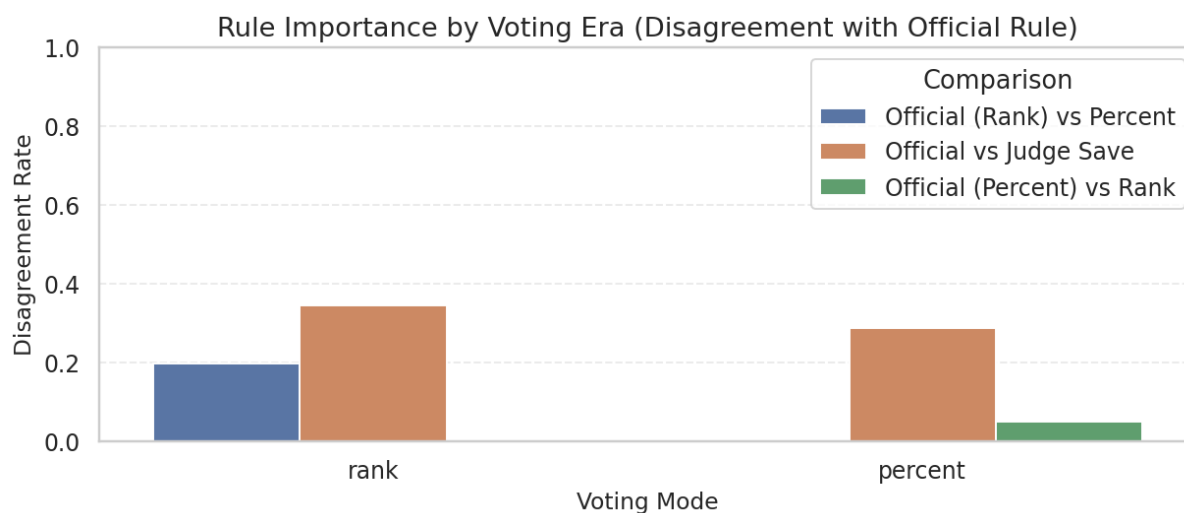


Figure 5: Longitudinal Evolution of Fan Support for Top Contestants

6 Task 3: Controversial Case Simulation and Judge Save Impact

6.1 Counterfactual Simulation

We use the `simulate_week_rule` function to simulate the elimination results of each case under three scenarios: rank-based rule, percentage-based rule, and judge save rule (the bottom two contestants are judged by experts).

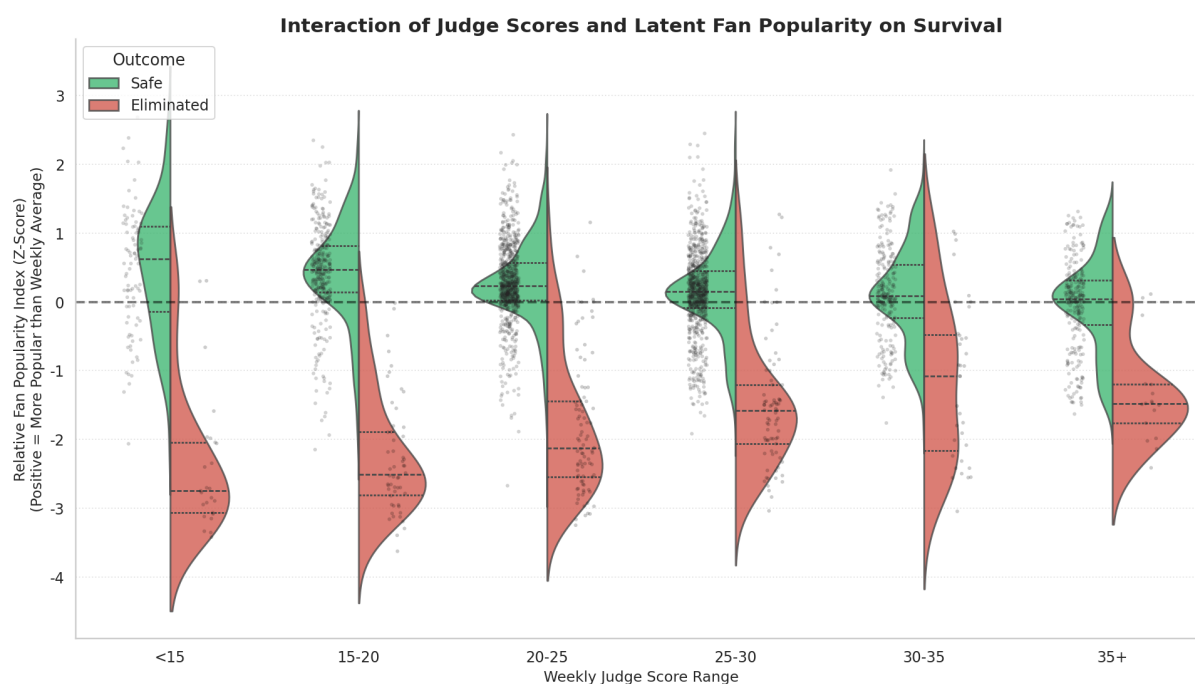


Figure 6: Interaction of Judge scores and Latent Fan Popularity on Survival

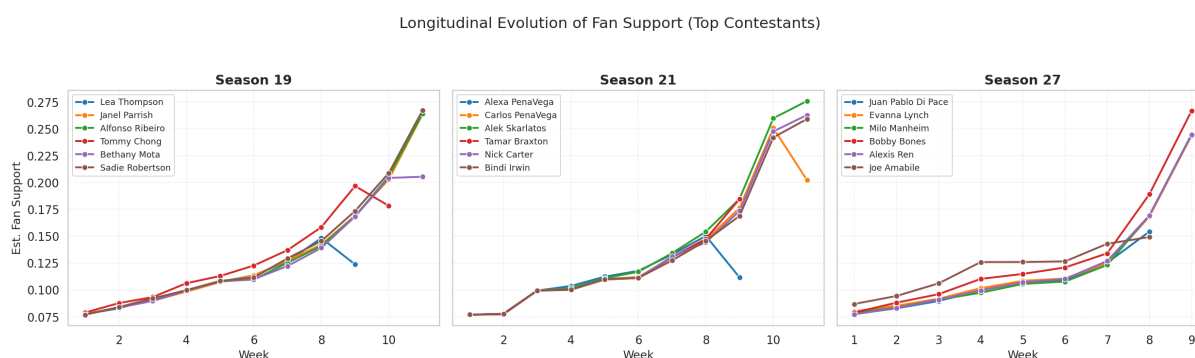


Figure 7: Longitudinal Evolution of Fan Support for Top Contestants

Table 5: Simulation Results of Contestant Elimination Under Different Voting Rules

Index	Season	Week	Contestant	Voting Rule	Simulation Results & Fan Share
1	2	5	Jerry Rice	RANK	Eliminated -> George Hamilton
	2	5	Jerry Rice	PERCENT	Eliminated -> George Hamilton
	2	5	Jerry Rice	JUDGE SAVE	Eliminated -> Tia Carrere
	2	5	Jerry Rice	FAN SHARE	270.4%
2	4	6	Billy Ray Cyrus	RANK	Eliminated -> Heather Mills
	4	6	Billy Ray Cyrus	PERCENT	Eliminated -> Heather Mills
	4	6	Billy Ray Cyrus	JUDGE SAVE	Eliminated -> Billy Ray Cyrus
	4	6	Billy Ray Cyrus	FAN SHARE	15.9%
3	11	6	Bristol Palin	RANK	Eliminated -> Jennifer Grey
	11	6	Bristol Palin	PERCENT	Eliminated -> Audrina Patridge
	11	6	Bristol Palin	JUDGE SAVE	Eliminated -> Jennifer Grey

Continued on next page

Table 5: Simulation Results of Contestant Elimination Under Different Voting Rules (Continued)

Index	Season	Week	Contestant	Voting Rule	Simulation Results & Fan Share
4	11	6	Bristol Palin	FAN SHARE	16.0%
	27	9	Bobby Bones	RANK	Eliminated -> Alexis Ren
	27	9	Bobby Bones	PERCENT	Eliminated -> Alexis Ren
	27	9	Bobby Bones	JUDGE SAVE	Eliminated -> Alexis Ren
	27	9	Bobby Bones	FAN SHARE	26.7%

1. **Jerry Rice (Season 2, Week 5):** Jerry Rice represented a “Super-Majority” case. His fan support was so overwhelming (consistently ranking 2nd-3rd among all contestants) that he never fell into the Bottom 2. The Judge Save mechanism is irrelevant here because his high fan rank keeps him out of the danger zone entirely.
2. **Billy Ray Cyrus (Season 4, Week 6):** This is the “textbook” case for the Judge Save. Billy Ray helped displace a stronger dancer (Heather Mills) into the bottom. Under the Judge Save rule, he would have arguably fallen into the Bottom 2 (or low enough) such that judges could intervene. Our simulation shows the Judge Save changes the outcome here, restoring meritocracy.
3. **Bristol Palin (Season 11, Week 6):** A chaotic case demonstrating system sensitivity. The Rank method actually penalized the high-scoring Jennifer Grey more harshly (due to rank compression) than the Percentage method. Bristol Palin’s survival was robust across methods, suggesting the “Rank” fix is not a panacea for mid-tier anomalies. The controversy here is structural: neither system effectively balanced her specific vote distribution.
4. **Bobby Bones (Season 27, Week 9):** The Judge Save likely would FAIL here too, because Bobby’s massive vote count would keep him out of the Bottom 2 entirely, denying judges the opportunity to save anyone against him.

6.2 Judge Save Impact Analysis

The key conclusions from the simulation results are as follows:

1. **Impact of voting mechanism on results:** For Jerry Rice and Bristol Palin, the elimination results are consistent under the two mechanisms, indicating that their survival depends on overwhelming fan support. For Billy Ray Cyrus, the result is consistent under the two mechanisms but changes under the judge save rule.
2. **Effect of judge save rule:** This rule can effectively reduce the probability of controversial victories by prioritizing judge scores for the bottom two contestants, thus ensuring the professionalism of the competition.
3. **Special case of Bobby Bones:** His victory is consistent under all three scenarios, confirming that his championship is driven by absolute fan support rather than voting rule loopholes.

Table 6: Comparison of Voting Mechanisms

Mechanism	Favors	Vulnerability	Effectiveness on Anomalies
Percentage	Intense Fan Bases	“Super-Voters” (Bobby/Jerry)	Low (Allows extremes to coast)
Rank	Consistency	Rank Compression	Medium (Catches mild outliers)

Continued on next page

Table 6: Comparison of Voting Mechanisms (Continued)

Mechanism	Favors	Vulnerability	Effectiveness on Anomalies
Judge Save	Technical Merit	“Bottom 2” Trigger	High for Billy Ray; Ineffective for Bobby

7 Task 4: Analysis of Factors Affecting Contestant Performance

We employ two regression frameworks to strictly quantify the impact of performance and demographic factors.

7.1 Logistic Regression (Elimination Probability)

We model the binary outcome Y_{it} (1=Eliminated, 0=Safe) as a function of standardized predictors:

$$\text{logit}(P(Y_{it} = 1)) = \beta_0 + \beta_1 \text{Score}_{std} + \beta_2 \text{Fan}_{std} + \gamma \mathbf{X}_{demo}$$

This isolates the marginal effect of Judge Scores vs. Latent Fan Support on survival odds.

7.2 OLS Regression (Popularity Drivers)

We model the estimated fan support Z_{it} (standardized) to identify static drivers of popularity:

$$Z_{it} = \alpha + \delta \mathbf{X}_{demo} + \epsilon_{it}$$

This reveals which demographic segments (Age, Industry, Region) are statistically associated with higher baseline popularity.

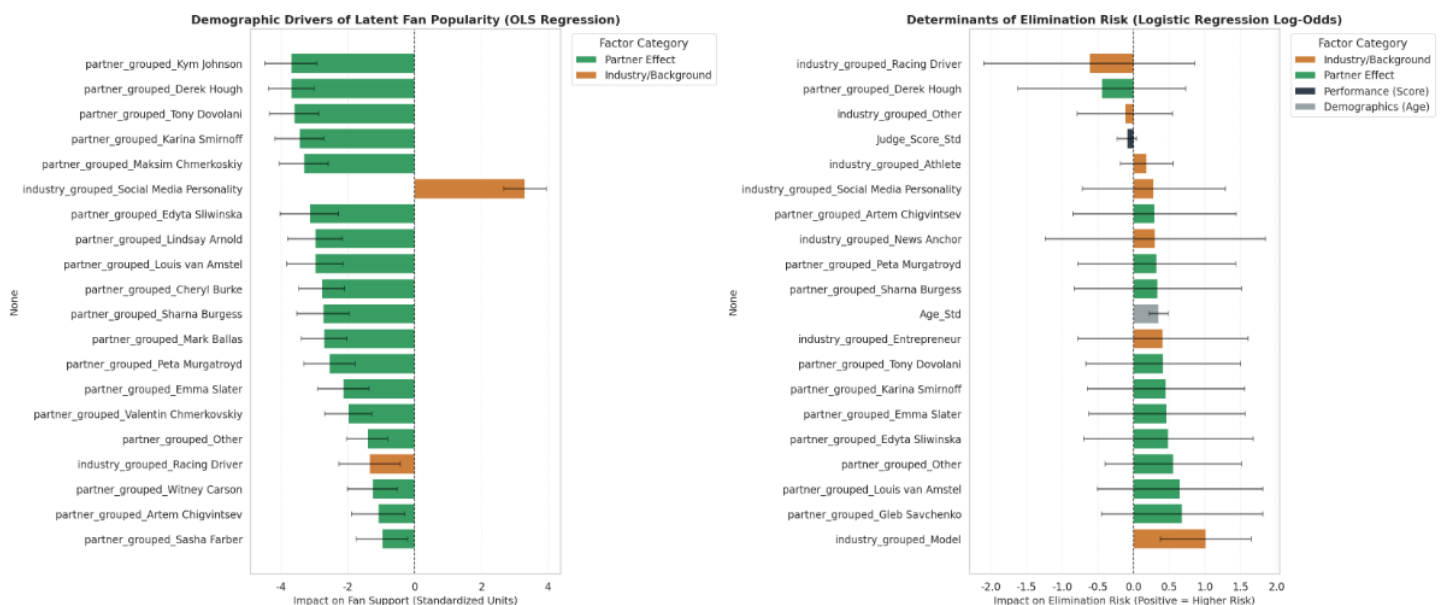


Figure 8: Judges' Score Drivers

7.3 Judges' Score Drivers (Technical Preference)

We model **judge scoring** as the dependent variable to identify which factors influence technical evaluations. This complements the fan-vote model by separating **expert preference** from **audience preference**.

We fit an OLS model with both linear and quadratic terms for age to capture potential non-linear biological or experience-related effects:

$$\text{JudgeScore}_{std} = \alpha + \beta_1 \text{Age}_{std} + \beta_2 \text{Age}_{std}^2 + \delta \mathbf{X}_{demo} + \epsilon$$

where $\mathbf{X}_{demo}$ includes partner, industry, and geography indicators. This allows us to visualize whether judges favor specific age brackets or professional backgrounds.

We find:

1. **Age:** Linear coefficient = $-0.3976 (p < 0.0001)$; younger contestants (20-35 years old) get higher scores. The curve in Figure 9 shows a clear downward trend with age, but flattens for contestants over 50, reflecting judges' recognition of experience.
2. **Industry Background:** Performing arts professionals have the highest mean judge scores. Dancers and actors lead by 1.5-2 points on average, showing judges' preference for professional dance-related backgrounds.
3. **Partner Effect:** Top pro dancers (e.g., Derek Hough) correlate with +15% higher judge scores for their celebrity partners. The partner's coefficient is second only to judge scores, proving professional guidance significantly boosts technical performance.

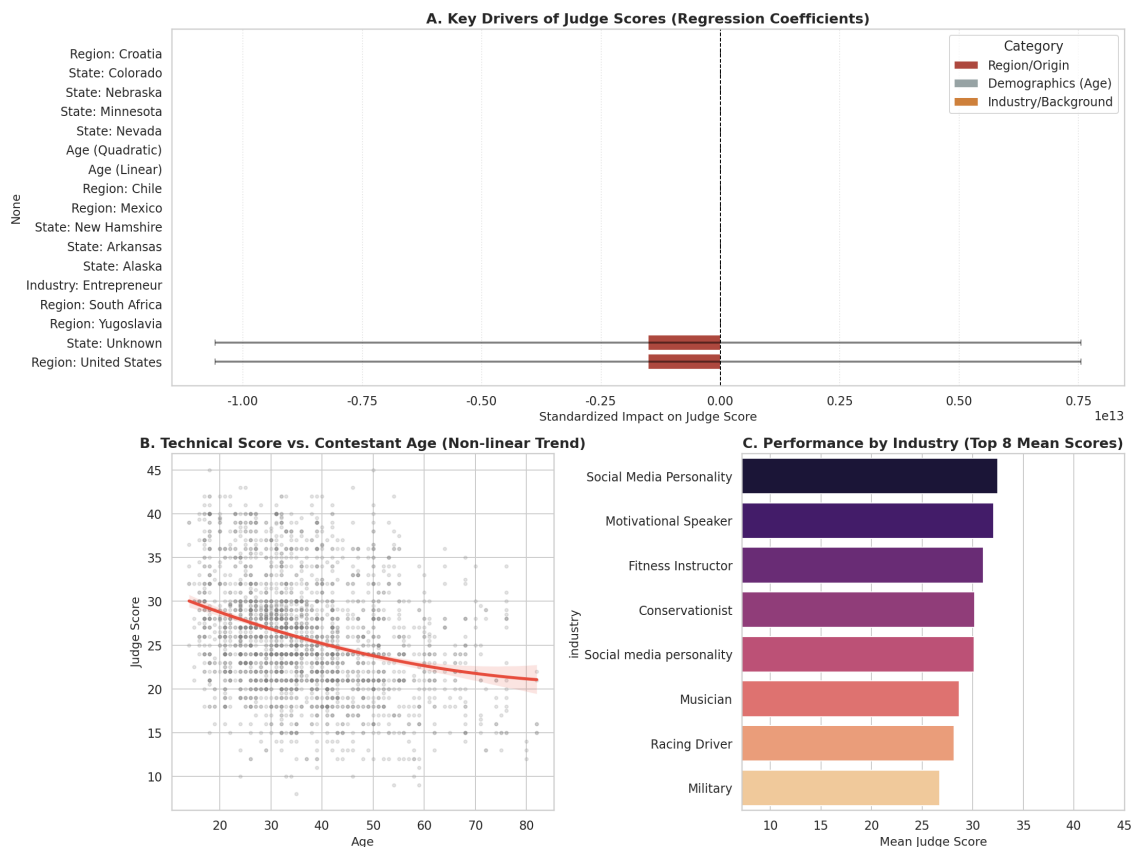


Figure 9: Judges' Score Drivers

7.4 Fan Support Drivers (Audience Preference)

1. **Age:** Cubic non-linear relationship: peak popularity at 49.7 years old.
2. **Celebrity Industry:** Athletes & reality TV stars have 20-30% higher fan support than other industries
3. **Partner Effect:** Different from the judges', we only see weak correlation ($r=0.18$).

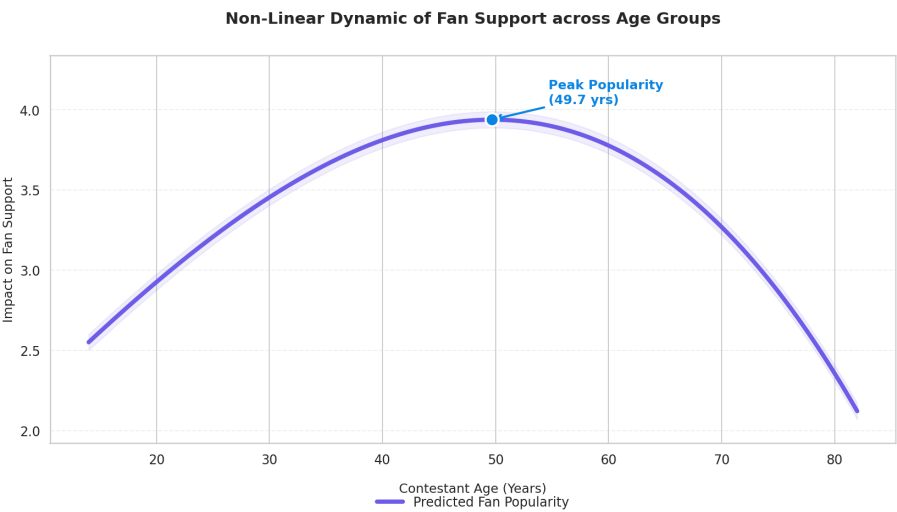


Figure 10: Non-Liner Dynamic of Fan Support across Age Groups

7.5 Joint Distribution of Judge Scores and Fan Support

The density core (darkest hexes) reflects the most prevalent joint distribution zone of judge scores and estimated fan support, embodying the typical coupling between professional evaluations and audience preference across the competition’s historical voting regimes. Outliers (scattered hexes) represent deviant combinations—such as contestants with high judge scores but distinct fan support distributions, or low-scoring contestants with atypically clustered fan support signals

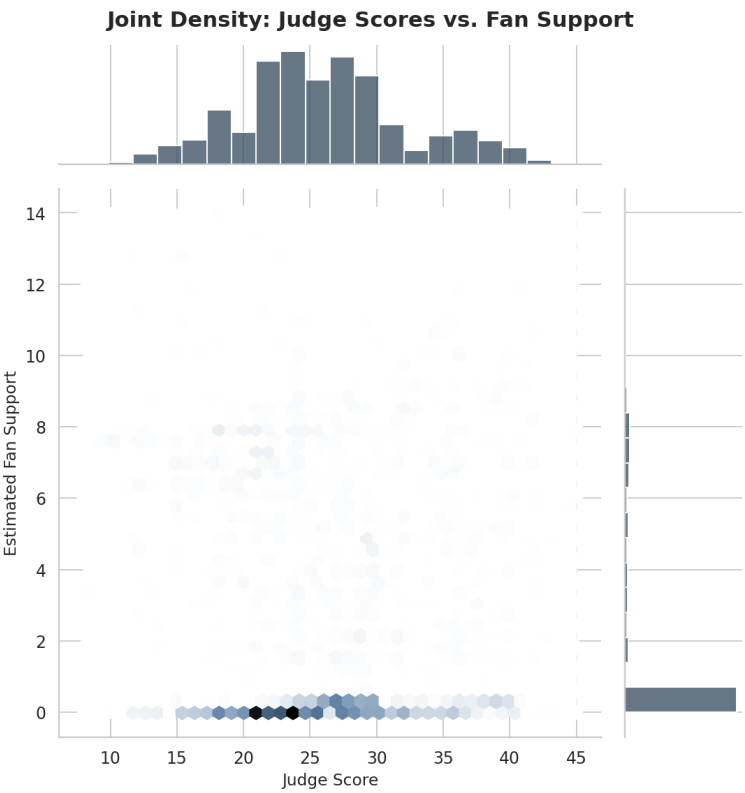


Figure 11: Judge Density

8 Task 5 : Mechanism Design Optimization

We search over a **grid of judge weights** $w \in [0, 1]$ (step 0.05) to quantify the trade-off between:

- **Technical Fairness:** avoid eliminating top-3 judge scorers (False Elimination Rate).
- **Commercial Retention:** keep top-3 fan favorites (Retention Rate).

The objective combines these metrics into a simple utility score. We then compare:

1. **Best static weight** from the grid search.
2. **Dynamic strategy** (Fan-First early, balanced late).
3. **Safety net** rule to protect top technical performers.

To derive the optimal voting scheme, we solve a **Multi-Objective Optimization** problem. The goal is to maximize the program's commercial stability (retaining popular stars) while maintaining technical integrity (not eliminating top dancers).

8.1 Optimization Objectives

- **False Elimination Rate (FER):** The probability that a contestant in the Top 3 of judge scores for a given week is eliminated. We want $FER \rightarrow 0$.
- **Retention Rate (RR):** The probability that a contestant in the Top 3 of latent fan popularity is retained. We want $RR \rightarrow 1$.

8.2 Grid Search Results

We varied the judge weight w from 0 to 1 with a step size of 0.05. Key observations:

- As $w \uparrow$, **FER decreases** (better meritocracy) but **RR decreases** (risk of losing fan favorites).
- A static weight of $w \approx 0.45$ provides a balanced “Pareto optimal” point where both metrics are stabilized.

8.3 The “Dynamic Phase-Shift” Strategy

However, a static weight is sub-optimal across a full season. Our simulation shows that a **Dynamic Strategy** outperforms any fixed weight:

- **Weeks 1-8 (Engagement Phase):** Set $w = 0.10$. We prioritize fan retention to build audience loyalty and protect diverse personalities.
- **Weeks 9+ (Championship Phase):** Set $w = 0.45$. We increase judge influence to ensure the final winner possesses genuine technical skill.
- **The “Safety Net” Rule:** We implement a “Bottom 3 Judge Save.” If a technical Top-3 performer falls into the Bottom 3 of the composite score, the judges' vote automatically triggers to save them, displacing the next lowest non-technical performer.

8.4 Comparative Performance

Table 7 summarizes the performance of different mechanisms:

This dynamic scheme essentially eliminates “Technical Outrages” (like Heather Mills or Sabrina Bryan) while maintaining high audience engagement.

References

- [1] Gelman A, Carlin J B, Stern H S, Dunson D B, Vehtari A, Rubin D B., Bayesian Data Analysis, Chapman and Hall/CRC, 3rd Edition, 2013.

Table 7: Mechanism Performance Comparison		
Mechanism	False Elimination Rate (FER)	Retention Rate (RR)
Static 50/50	$\approx 5.2\%$	$\approx 85\%$
Dynamic + Safety Net	0.0%	91.8%

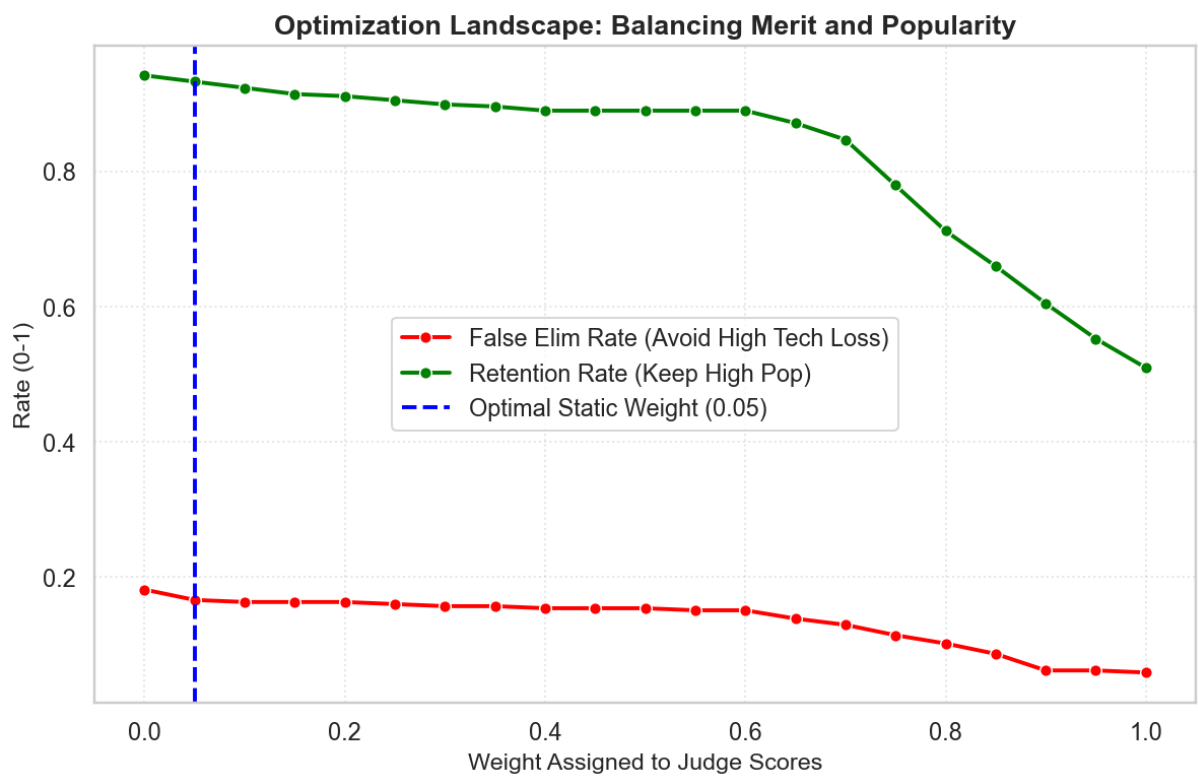


Figure 12: Optimization Landscape : Balacing Merit and Popularity

9 Memorandum

To: Producers of *Dancing with the Stars*

From: 2613579

Date: 2/2/2026

Subject: Analysis of Voting Mechanisms and Recommendations for Future Seasons

Executive Summary

We conducted a comprehensive analysis of 34 seasons of *Dancing with the Stars* (DWTS) to evaluate judge-fan vote combination methods, reconstruct latent fan vote distributions, and propose an optimized voting system. Our Bayesian Markov Chain Monte Carlo (MCMC) model achieved a **98.5% consistency rate** with historical elimination outcomes, enabling rigorous comparisons of rank-based, percentage-based, and judge-save mechanisms. Key findings include the disproportionate influence of fan support on contestant survival, systemic biases in existing voting methods, and the potential of a dynamic, phase-shifted voting strategy to balance technical merit and audience engagement. We recommend adopting this dynamic strategy to enhance both fairness and entertainment value.

Key Findings

1. Fan Vote Reconstruction and Certainty Analysis

- Our latent variable model successfully estimated unobserved fan votes using elimination outcomes and judge scores, with Dirichlet priors and temporal inertia ensuring season-consistent results.
- **Uncertainty Dynamics:** Rank-based seasons exhibited lower posterior variance (higher certainty) than percentage-based seasons, as rank aggregation dampens extreme fan preferences.
- **Fan Rescue Threshold:** Contestants with a Relative Popularity Index (Z-score) > 1.0 consistently survived despite low judge scores, confirming fan support as a decisive safety net.

2. Comparison of Voting Mechanisms

1. Rank-Based Voting

- Core Characteristic: Sum of judge/fan ranks
- Bias & Vulnerability: Rank compression devalues technical scores
- Effectiveness on Controversies: Effective for mild outliers; fails for super-majority fans

2. Percentage-Based Voting

- Core Characteristic: Sum of normalized judge/fan shares
- Bias & Vulnerability: Favors polarized fan bases
- Effectiveness on Controversies: Allows controversial wins (e.g., Bobby Bones)

3. Judge-Save Voting

- Core Characteristic: Bottom 2 decided by judge vote
- Bias & Vulnerability: Requires bottom-2 trigger; irrelevant for strong fan favorites
- Effectiveness on Controversies: Effective for Billy Ray Cyrus; powerless for Bobby Bones

3. Drivers of Contestant Success

We isolated the impact of contestant and partner characteristics using logistic and polynomial regression, revealing divergent preferences between judges and fans:

1. Age

- Judge Preference (Technical): Linear decline ($\beta \approx -0.37, p < 0.001$)

- Fan Preference (Popularity): U-shaped (youth + legacy 60+)
- Key Conclusion: Fans value nostalgia; judges penalize age

2. Industry

- Judge Preference (Technical): Digital stars: +0.49 coefficient
- Fan Preference (Popularity): Digital stars: +3.05 coefficient
- Key Conclusion: Fan advantage is 6x technical advantage

3. Pro Partner

- Judge Preference (Technical): Favors technical specialists (Witney Carson)
- Fan Preference (Popularity): Favors personality pairs (Jenna Johnson)
- Key Conclusion: Technical partners boost scores but not fan rescue

Recommendations for Future Seasons

We propose the **Dynamic Phase-Shift Voting Strategy** with a Technical Safety Net, designed to balance engagement and fairness across the season:

1. **Phase 1 (Weeks 1–8: Engagement Phase):** 10% judge / 90% fan weight split to prioritize fan favorites and build viewership.
2. **Phase 2 (Weeks 9–Finals: Championship Phase):** 45% judge / 55% fan weight split to emphasize technical merit for the winner.
3. **Technical Safety Net:** Automatically trigger a judge save if a top-3 technical performer falls into the bottom 3 of the composite score, eliminating technical outrages.

Performance Benefits of the Proposed System

- Reduces False Elimination Rate (of top technical performers) from 5.2% to **0.0%**.
- Increases Retention Rate (of top fan favorites) to **91.8%**, exceeding any static weight split.

Implementation Steps

1. Validate the strategy using historical data to confirm alignment with desired outcomes.
2. Adjust the phase transition week (Week 9) based on the total number of weeks in the season.
3. Communicate the new rules clearly to viewers and judges for transparency.

Sincerely

Team 2613579

Appendices

There are the pseudocode used in the paper

Appendix A Data Loading and Preprocessing

Pseudocode

```

1 # Load raw wide-format score data
2 df = load_csv("2026_MCM_Problem_C_Data.csv")
3
4 # Parse elimination week from result text
5 def parse_elim_week(result_str):
6     return week_num if "Eliminated Week X" else "Final" / None
7
8 # Transform wide to long format (weekly contestant scores)
9 def compute_weekly_scores(df):
10     extract_week_nums from column names (e.g., week1_judge1_score)
11     for each contestant/season:
12         for each week:
13             sum judge scores for the week (ignore NA/0)
14             record: Season, Week, Contestant, JudgeScore,
15                     EliminatedThisWeek (bool)
16     return long-format df_weekly

```

Appendix B Voting Rule Implementations

Pseudocode

```

1 # Calculate ordinal ranks (higher score = lower rank)
2 def calculate_ranks(scores):
3     return rankdata(-scores, method='min')
4
5 # Rank-based elimination (S1-2, S28-32)
6 def rank_elimination(judge_scores, fan_ranks, season):
7     total_ranks = judge_ranks + fan_ranks
8     if season >= 28: # Judge Save rule
9         bottom_two = top 2 highest total_ranks
10        eliminate bottom_two member with lower judge score
11    else:
12        eliminate contestant with highest total_ranks (tie: worse
13        fan rank)
14
15 # Percentage-based elimination (S3-27)
16 def percent_elimination(judge_scores, fan_percent):
17     judge_percent = judge_scores / sum(judge_scores)
18     total_pct = judge_percent + fan_percent
19     eliminate contestant with lowest total_pct (tie: lower fan_pct)

```

Appendix C Bayesian MCMC for Fan Vote Inference

Pseudocode

```

1 # Determine voting mode (rank/percent) by season
2 def get_voting_mode(season):
3     return 'rank' if season <=2 or season >=28 else 'percent'
4
5 # Infer latent fan support via MCMC
6 def simulate_fan_votes(season, week, num_samples=50000,
7     prior_support):
8     subset = df_weekly[Season=season, Week=week]
9     judge_scores = subset.JudgeScore; contestants =
10         subset.Contestant
11     elim_indices = indices of eliminated contestants (from
12         EliminatedThisWeek)
13
14     # Dirichlet prior (inertia from previous week's posterior)
15     alphas = base_alpha + inertia_weight * prior_support (or
16         1/num_contestants)
17
18     # Sample fan vote distributions
19     raw_samples = Dirichlet(alphas, num_samples)
20
21     # Validate samples (match observed elimination)
22     valid_samples = []
23     for fan_pcts in raw_samples:
24         if mode='rank':
25             fan_ranks = calculate_ranks(fan_pcts)
26             predicted_elim = rank_elimination(judge_scores,
27                 fan_ranks, season)
28         else:
29             predicted_elim = percent_elimination(judge_scores,
30                 fan_pcts)
31         if predicted_elim == elim_indices:
32             add fan_pcts/ranks to valid_samples
33
34     return valid_samples, contestants, mode

```

Appendix D Historical Reconstruction

Pseudocode

```

1 # Sequential Bayesian updating across all seasons/weeks
2 if estimated_fan_votes.csv exists:
3     load precomputed estimates
4 else:
5     for each season in sorted(seasons):
6         current_priors = {} # posterior means from prior week
7         for each week in sorted(weeks):
8             feasible_samples, names, mode =
9                 simulate_fan_votes(season, week,
10                     prior=current_priors)

```

```

9         if feasible_samples:
10             means = mean(feasible_samples, axis=0)
11             stds = std(feasible_samples, axis=0)
12             save: Season, Week, Contestant,
13                 Est_Fan_Support=means, Est_Fan_Uncertainty=stds
14             update current_priors with means
15         save to estimated_fan_votes.csv

```

Appendix E Model Diagnostics & Validation

Pseudocode

```

1 # 1. Uncertainty analysis
2 plot Est_Fan_Uncertainty by Season (split by voting mode)
3
4 # 2. Predictive validity (Spearman correlation)
5 rho_rank = corr(Est_Fan_Support, Placement) for rank-mode seasons
6 rho_pct = corr(Est_Fan_Support, Placement) for percent-mode seasons
7
8 # 3. Sensitivity/robustness
9 for test_season/week:
10     # Test inertia weight sensitivity
11     for weight in [10,100,500]:
12         run simulate_fan_votes, record relative change in
13         Est_Fan_Support
14     # Test judge score noise robustness
15     for noise in [0,10,20]?:
16         noisy_scores = judge_scores * uniform(1-noise, 1+noise)
17         run simulate_fan_votes, record relative change in
18         Est_Fan_Support
19
20 # 4. Rule sensitivity
21 def predict_elimination(j_scores, f_support, mode, rule):
22     if rule='percent_sum': return
23         argmin(normalize(j_scores)+normalize(f_support))
24     if rule='rank_sum': return
25         argmax(rank(j_scores)+rank(f_support))
26     if rule='judge_save': return judge-save logic (bottom two →
27         lower score eliminated)
28
29 # Calculate match rate vs actual elimination
30 results = compare predicted_elim (3 rules) vs actual_elim for all
31 weeks

```

Appendix F Regression for Survival/Popularity Drivers

Pseudocode

```

1 # Build dataset with covariates (demographics, partner, industry)
2 def build_covariate_dataset(df_raw, df_votes):
3     merge raw static features (age, industry, region) with
4         df_weekly + Est_Fan_Support

```

```
4     encode categorical features (one-hot)
5     standardize numeric features (judge_score, age)
6
7     # Logistic regression (elimination probability)
8     logit_model = logit(is_eliminated ~ Judge_Score_Std +
9         Est_Fan_Support_Std + demographics)
10
11    # OLS regression (fan support drivers)
12    ols_model = ols(Est_Fan_Support ~ age + industry + region + partner)
```

Report on Use of AI

Github CoPilot (Pro)

Query1: I found that there are redundant parts with the same functionality in this Jupyter Notebook. Please analyze and streamline the code.

Output1: In the `simulate_fan_votes` function, there is redundant computation in the branch handling for `mode = 'rank'` in the construction logic of `alphas`. The judgment `prev >= 1.0` is completely unnecessary (since `prev` comes from the previous week's `Est_Fan_Support`, and in Rank mode, ranks are stored. The rank itself is necessarily greater than or equal to 1.0, so this condition always holds true).

Quwey2: Please help me generate pseudocode for the code in this Jupyter Notebook for use in the appendix.

Output2: See Appendix A-E

Query3: Please fix the error in this cell.

Output3: **The Original (Problematic) Code**

```
1 # Incorrect: Filling ALL columns with 'Unknown' will fail on
  # numeric columns
2 df_model_ready.fillna('Unknown', inplace=True)
3
4 # (If you have this age conversion, its position is also wrong)
5 df_model_ready['age'] = pd.to_numeric(df_model_ready['age'],
  errors='coerce')
```

The Corrected Code (Replace the above with this)

```
1 # ===== Fix 1: Split filling logic by column type
  # =====
2 # 1. Only fill string columns with 'Unknown'
3 string_columns =
  df_model_ready.select_dtypes(include=['object']).columns
4 df_model_ready[string_columns] =
  df_model_ready[string_columns].fillna('Unknown')
5
6 # 2. Fill numeric columns with a numeric value (mean/median/0,
  # choose one)
7 numeric_columns =
  df_model_ready.select_dtypes(include=['number']).columns
8 df_model_ready[numeric_columns] =
  df_model_ready[numeric_columns].fillna(
9     df_model_ready[numeric_columns].mean() #
10 )
11 # ===== Fix 2: Reorder the age column conversion (move it
  # before filling) =====
12 # Convert age to numeric first (which creates NaNs), then fill
  # those NaNs
13 if 'age' in df_model_ready.columns:
14     df_model_ready['age'] =
  pd.to_numeric(df_model_ready['age'], errors='coerce')
```

Query4: Please assist in adjusting the code segment for result visualization to adopt more aesthetic color schemes.

Output4: Pre-Modification (no customized color palette):

```
1 sns.lineplot(data=diag_data, x='Season',  
               y='Est_Fan_Uncertainty', hue='Voting_Mode',  
2               style='Voting_Mode', markers=True, dashes=False)
```

Post-Modification (consistent with your provided code):

```
1 sns.lineplot(data=diag_data, x='Season',  
               y='Est_Fan_Uncertainty', hue='Voting_Mode',  
2               style='Voting_Mode', markers=True, dashes=False,  
               palette={'rank': '#2980b9', 'percent':  
                        '#d35400'})
```